

Optimization Pipeline – Overview and Hamiltonian Engine Guide

1. Overview

This repository implements a **modular optimization pipeline** designed to transform a high-level optimization problem definition into a **Hamiltonian representation suitable for QUBO/Ising-based quantum or classical optimization solvers**.

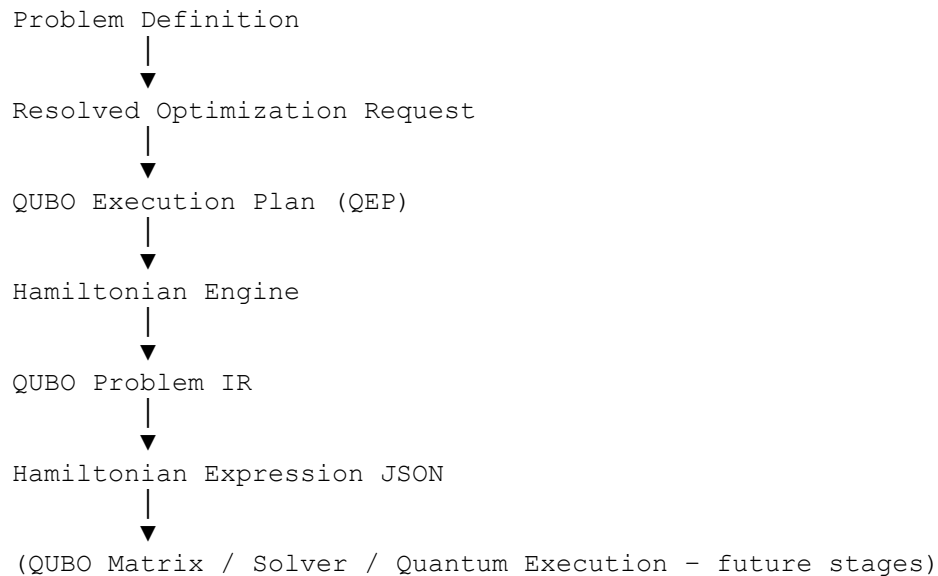
The pipeline separates **problem specification**, **compilation**, and **Hamiltonian generation** to ensure:

- Clear modular architecture
- Reusable components
- Domain-independent optimization processing

The system supports multiple optimization problems (e.g., **Travel Sequence Optimization**, **Portfolio Optimization**) while using the same core pipeline.

2. Optimization Pipeline Architecture

The pipeline consists of the following major stages:



Stage Descriptions

1. Problem Definition

A structured optimization request describing decision variables, constraints, datasets, and objectives.

2. Resolved Optimization Request

The validated and fully expanded version of the optimization request.

3. QUBO Execution Plan (QEP)

A staged plan defining how the optimization problem is compiled into QUBO form.

4. Hamiltonian Engine

The component responsible for transforming the QEP into a **QUBO Intermediate Representation (IR)**.

5. QUBO IR

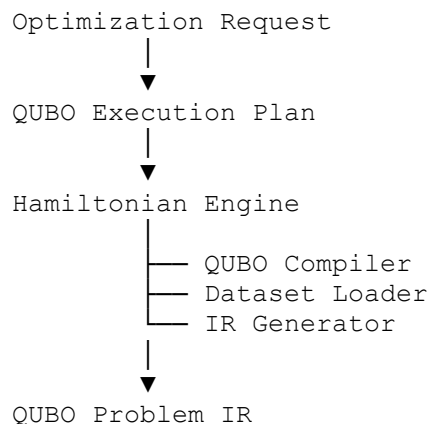
A structured representation of the Hamiltonian problem, suitable for further transformations.

6. Hamiltonian Expression Generation

Human-readable Hamiltonian expressions derived from the IR.

3. Where the Hamiltonian Engine Fits

The **Hamiltonian Engine** sits at the **core compilation stage** of the optimization pipeline.



Responsibilities of the Hamiltonian Engine

The engine performs the following tasks:

- Loads problem datasets
- Processes the **QUBO Execution Plan**
- Compiles each stage into a **QUBO Intermediate Representation**
- Generates the **Hamiltonian IR file**
- Optionally generates a **human-readable Hamiltonian JSON**

Important Design Principles

The Hamiltonian Engine strictly enforces:

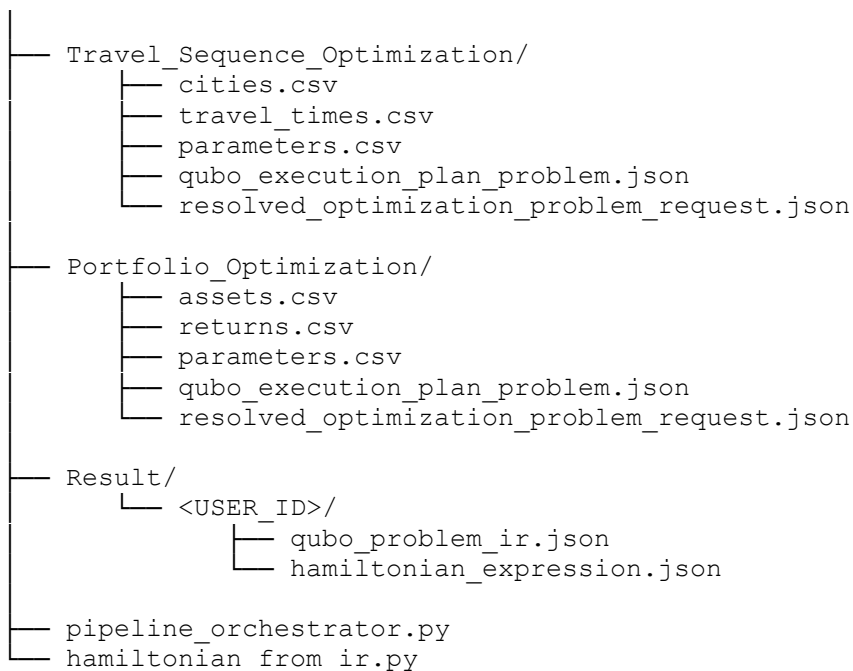
- **NO domain logic**
- **NO solver logic**
- **NO AI / GenAI logic**

It functions purely as a **compiler layer** within the optimization system.

4. Directory Structure

Example directory layout:

Hamiltonian_Engine/



5. Running the Optimization Pipeline

Step 1 — Start the Pipeline

Run the pipeline orchestrator:

```
python pipeline_orchestrator.py
```

Step 2 — Enter User ID

You will be prompted to enter a **4-digit User ID**.

Example:

Enter 4-digit USER ID: 1001

This ID is used to create a unique result folder:

Result/1001/

Step 3 — Select Optimization Problem

Choose the optimization problem type:

1 → Travel Sequence Optimization
2 → Portfolio Optimization

Example:

Enter choice (1 or 2): 1

Step 4 — QUBO IR Generation

The pipeline will:

- Load the problem datasets
- Compile the QUBO Execution Plan
- Generate the **QUBO Problem IR**

The IR file will be created at:

Result/<USER_ID>/qubo_problem_ir.json

Step 5 — Optional Hamiltonian Generation

After IR creation, the system will prompt:

Generate complete Hamiltonian JSON file? (Y/N)

If **Y** is selected:

- The Hamiltonian generator will run
- A human-readable Hamiltonian file will be created

Result/<USER_ID>/hamiltonian_expression.json

6. Example Output

Result/1001/

```
└─ qubo_problem_ir.json
└─ hamiltonian_expression.json
```

7. Future Extensions

The pipeline is designed to support future extensions such as:

- QUBO Matrix generation
 - Solver integration (classical / quantum)
 - QAOA or annealing execution
 - Solution decoding
 - Result contracts and analytics
-

8. Summary

The **Hamiltonian Engine** acts as a **universal compiler layer** that transforms optimization problems into Hamiltonian representations while remaining:

- domain-agnostic
- solver-agnostic
- extensible

This modular design allows the optimization pipeline to evolve while keeping the Hamiltonian compilation stage stable and reusable.